

VALIDACIONES EN WEB API

Guía general con C# / ASP.NET Core

Programación II

Models · Controllers · IActionResult · try/catch · LINQ · DTOs

Objetivo: que el estudiante pueda decidir qué validar, en qué momento hacerlo y qué respuesta HTTP devolver cuando un dato no cumple las condiciones esperadas.

1. ¿Qué significa validar?

Validar significa comprobar que los datos recibidos por la API sean correctos antes de utilizarlos. Una API no debería asumir que todo dato enviado por el cliente es válido.

Idea clave: validar no es solamente evitar errores. También permite que la API responda de manera clara y predecible ante datos incorrectos.

¿Qué puede llegar desde el cliente?

- Parámetros de ruta: /api/alumnos/15
- Parámetros de consulta: /api/alumnos?nombre=ana
- Datos enviados en el body de una petición POST o PUT.
- Identificadores relacionados: idMateria, idAlumno, idUsuario, etc.
- Colecciones: listas de IDs, elementos seleccionados o registros a procesar.

2. Un orden práctico para pensar las validaciones

Cuando llega una petición, conviene revisar los datos en un orden lógico. Esto evita mezclar problemas distintos y hace que el código sea más fácil de leer.

Paso	Pregunta	Ejemplo	Respuesta habitual
1	¿El dato llegó?	Nombre nulo o vacío	400 Bad Request
2	¿Tiene el tipo/formato correcto?	"hola" donde se espera un número	400 Bad Request
3	¿Cumple el rango o reglas básicas?	Edad negativa, texto demasiado corto	400 Bad Request
4	¿Existe el recurso relacionado?	idAlumno inexistente	404 Not Found
5	¿Se puede realizar la operación?	Inscripción duplicada	400 / 409 según el criterio
6	¿Ocurrió un error inesperado?	Excepción no prevista	500 Internal Server Error

3. Validaciones básicas

3.1. Validar un identificador numérico

Aunque el parámetro sea un entero, todavía debemos validar que tenga sentido para nuestra aplicación.

```
[HttpGet("{id}")]
public IActionResult GetById(int id)
{
    if (id <= 0)
        return BadRequest("El id debe ser mayor a 0.");

    // continuar con la búsqueda...
    return Ok();
}
```

Atención: si el proyecto usa [ApiController], ASP.NET Core puede rechazar automáticamente algunos errores de conversión de parámetros. Aun así, las reglas como `id > 0` siguen siendo responsabilidad de nuestra aplicación.

3.2. Validar texto nulo, vacío o con espacios

```
if (string.IsNullOrEmpty(alumno.Nombre))
{
    return BadRequest("El nombre es obligatorio.");
}
```

`string.IsNullOrEmpty(...)` controla tres situaciones: null, cadena vacía (""), y cadenas formadas solamente por espacios.

3.3. Validar longitud

```
if (alumno.Nombre.Length < 2 || alumno.Nombre.Length > 50)
{
    return BadRequest("El nombre debe tener entre 2 y 50 caracteres.");
}
```

3.4. Validar rangos numéricos

```
if (alumno.Edad < 16 || alumno.Edad > 100)
{
    return BadRequest("La edad debe estar entre 16 y 100 años.");
}
```

3.5. Validar valores permitidos

```
if (turno != "Mañana" && turno != "Tarde" && turno != "Noche")
{
    return BadRequest("El turno indicado no es válido.");
}
```

Mejora posible: cuando los valores posibles son fijos, más adelante puede convenir usar enum en lugar de comparar strings.

4. Validar conversiones con TryParse

Cuando recibimos un dato como texto y necesitamos convertirlo, TryParse es preferible a Parse si el dato puede ser incorrecto, porque permite validar sin provocar una excepción.

```
[HttpGet("buscar/{idTexto}")]
public IActionResult Buscar(string idTexto)
{
    if (!int.TryParse(idTexto, out int id))
        return BadRequest("El id debe ser un número entero.");

    if (id <= 0)
        return BadRequest("El id debe ser mayor a 0.");

    return Ok(id);
}
```

Parse vs TryParse

	Parse	TryParse
Si el dato es válido	Convierte	Convierte
Si el dato es inválido	Lanza una excepción	Devuelve false
Uso recomendado	Datos controlados	Datos ingresados por el usuario o cliente

5. Validar existencia de registros

Antes de modificar, eliminar o relacionar un recurso, primero debemos comprobar que exista.

```
Alumno? alumno = Alumnos.FirstOrDefault(a => a.Id == id);

if (alumno == null)
{
    return NotFound("No se encontró el alumno.");
}
```

Esta validación es especialmente importante en GET por id, PUT, DELETE y operaciones que relacionan entidades.

6. Validar datos duplicados

Hay situaciones donde un dato puede ser correcto en formato, pero no debe repetirse.

```
bool existeLegajo = Alumnos.Any(a => a.Legajo == nuevoAlumno.Legajo);

if (existeLegajo)
```

```
{
    return BadRequest("Ya existe un alumno con ese legajo.");
}
```

LINQ como ayuda: Any, FirstOrDefault, Count y Where suelen aparecer dentro de las validaciones porque permiten consultar colecciones de forma clara.

7. Validar relaciones entre entidades

Si una operación recibe IDs de otras entidades, no alcanza con validar que sean mayores a cero. También debemos comprobar que esas entidades existan.

```
Materia? materia = Materias.FirstOrDefault(m => m.Id == inscripcion.IdMateria);
Alumno? alumno = Alumnos.FirstOrDefault(a => a.Id == inscripcion.IdAlumno);

if (materia == null)
    return NotFound("La materia no existe.");

if (alumno == null)
    return NotFound("El alumno no existe.");
```

Validar una relación duplicada

```
bool yaInscripto = Incripciones.Any(i =>
    i.IdAlumno == inscripcion.IdAlumno &&
    i.IdMateria == inscripcion.IdMateria);

if (yaInscripto)
{
    return BadRequest("El alumno ya está inscripto en esa materia.");
}
```

8. Validar colecciones

```
if (idsAlumnos == null || idsAlumnos.Count == 0)
{
    return BadRequest("Debe enviar al menos un alumno.");
}

if (idsAlumnos.Any(id => id <= 0))
{
    return BadRequest("Todos los ids deben ser mayores a 0.");
}
```

8.1. Validaciones que dependen de más de un dato

Algunas reglas no pueden comprobarse mirando un solo campo.

```
if (reserva.FechaFin <= reserva.FechaInicio)
{
    return BadRequest("La fecha de finalización debe ser posterior al inicio.");
}

if (asistencia.Presente && asistencia.Justificada)
{
    return BadRequest("Una asistencia no puede estar presente y justificada al mismo tiempo.");
}
```

9. Validaciones en DTOs con Data Annotations

Ahora que aparecen los DTOs, algunas reglas simples pueden declararse directamente en sus propiedades. Esto reduce validaciones repetitivas en los Controllers.

```
using System.ComponentModel.DataAnnotations;

public class CrearAlumnoDto
{
    [Required(ErrorMessage = "El nombre es obligatorio.")]
    [StringLength(50, MinimumLength = 2,
        ErrorMessage = "El nombre debe tener entre 2 y 50 caracteres.")]
    public string Nombre { get; set; }
}
```

```

public string Nombre { get; set; } = string.Empty;

[Range(16, 100, ErrorMessage = "La edad debe estar entre 16 y 100 años.")]
public int Edad { get; set; }

[Required]
public string Legajo { get; set; } = string.Empty;
}

```

[ApiController]: en un Controller marcado con [ApiController], los errores de validación del modelo pueden producir automáticamente una respuesta 400 Bad Request antes de ejecutar el método del endpoint.

¿Qué conviene poner en Data Annotations?

- Campos obligatorios.
- Longitud mínima o máxima.
- Rangos numéricos.
- Formato de email, teléfono u otras reglas simples de formato.

En cambio, reglas que dependen de otras colecciones o del estado del sistema (por ejemplo, "el legajo no debe repetirse") normalmente deben comprobarse durante la operación.

10. try/catch NO reemplaza las validaciones

try/catch sirve para manejar errores excepcionales. No debería utilizarse como reemplazo de una validación que podemos realizar antes.

```

[HttpPost]
public IActionResult Post(CrearAlumnoDto dto)
{
    try
    {
        if (string.IsNullOrEmpty(dto.Nombre))
            return BadRequest("El nombre es obligatorio.");

        bool repetido = Alumnos.Any(a => a.Legajo == dto.Legajo);
        if (repetido)
            return BadRequest("El legajo ya existe.");

        // Crear y agregar alumno...
        return Created("", dto);
    }
    catch (Exception ex)
    {
        return StatusCode(500, "Ocurrió un error inesperado.");
    }
}

```

Regla práctica: si sabemos qué puede estar mal y podemos comprobarlo con un if, eso es una validación. Si ocurre algo inesperado durante la ejecución, ahí puede intervenir catch.

11. ¿Qué código HTTP devolver?

Código	Cuándo usarlo	Ejemplo
200 OK	La operación fue correcta y devolvemos información.	GET exitoso.
201 Created	Se creó un nuevo recurso.	POST exitoso.
204 No Content	La operación fue correcta y no hace falta devolver contenido.	DELETE o PUT exitoso.
400 Bad Request	Los datos enviados no cumplen una regla.	Id <= 0, campo vacío, rango inválido.
404 Not Found	El recurso solicitado o relacionado no existe.	Alumno inexistente.
409 Conflict	Existe un conflicto con el estado actual del recurso.	Puede usarse para ciertos duplicados.

500 Internal Server Error	Ocurrió un problema inesperado en el servidor.	Excepción no controlada por validaciones previas.
---------------------------	--	---

12. Ejemplo completo: inscripción de un alumno

Este ejemplo reúne varias validaciones de las que venimos trabajando: datos básicos, existencia, duplicados, cupo, LINQ, IActionResult y try/catch.

```
[HttpPost("inscribir")]
public IActionResult Inscribir(InscripcionDto dto)
{
    try
    {
        // 1. Validaciones básicas
        if (dto.IdAlumno <= 0 || dto.IdMateria <= 0)
            return BadRequest("Los ids deben ser mayores a 0.");

        // 2. Validar existencia
        Alumno? alumno = Alumnos.FirstOrDefault(a => a.Id == dto.IdAlumno);
        if (alumno == null)
            return NotFound("El alumno no existe.");

        Materia? materia = Materias.FirstOrDefault(m => m.Id == dto.IdMateria);
        if (materia == null)
            return NotFound("La materia no existe.");

        // 3. Validar duplicado
        bool yaInscripto = Inscripciones.Any(i =>
            i.IdAlumno == dto.IdAlumno &&
            i.IdMateria == dto.IdMateria);

        if (yaInscripto)
            return BadRequest("El alumno ya está inscripto.");

        // 4. Validar cupo
        int cantidadInscriptos = Inscripciones.Count(i =>
            i.IdMateria == dto.IdMateria);

        if (cantidadInscriptos >= materia.CupoMaximo)
            return BadRequest("La materia no tiene cupos disponibles.");

        // 5. Ejecutar la operación
        Inscripcion nueva = new Inscripcion
        {
            Id = Inscripciones.Count + 1,
            IdAlumno = dto.IdAlumno,
            IdMateria = dto.IdMateria
        };

        Inscripciones.Add(nueva);
        return Created("", nueva);
    }
    catch (Exception)
    {
        return StatusCode(500, "Ocurrió un error al realizar la inscripción.");
    }
}
```

13. ¿Dónde deberían vivir las validaciones?

En los ejercicios simples que utilizan solamente Models y Controllers, es normal que muchas validaciones estén dentro del Controller. Sin embargo, al trabajar con una arquitectura en capas conviene separar responsabilidades.

Lugar	Tipo de validación	Ejemplo
DTO / Modelo de entrada	Forma y reglas simples del dato	Required, StringLength, Range.
Controller	Validaciones ligadas a la petición HTTP	id > 0, parámetros presentes, respuesta 400/404.

Lógica / Servicio	Reglas del sistema o del negocio	Cupo disponible, inscripción duplicada, estado permitido.
Acceso a datos	Consulta y persistencia	Buscar, guardar, actualizar, eliminar.

Para esta etapa: pueden mantener la validación dentro de los Controllers y, cuando pasen completamente a la arquitectura en capas, mover las reglas de negocio a la capa lógica.

14. Errores frecuentes al validar

1. Validar después de utilizar el dato.
2. Usar Parse con datos ingresados por el usuario cuando TryParse sería más seguro.
3. Devolver siempre 500 aunque el error sea un dato inválido.
4. Usar try/catch para controlar situaciones que deberían resolverse con un if.
5. No comprobar si un recurso existe antes de modificarlo o eliminarlo.
6. Validar solamente el formato de un ID y no la existencia del registro relacionado.
7. Repetir la misma validación en muchos endpoints cuando podría estar en un DTO.
8. Devolver mensajes demasiado genéricos como "error" sin indicar qué dato debe corregirse.
9. Confiar únicamente en las validaciones del frontend. La API siempre debe validar por su cuenta.

15. Checklist para revisar un endpoint

Antes de considerar terminado un endpoint, el estudiante puede hacerse estas preguntas:

- ☐ ¿Los IDs deben ser mayores a cero?
- ☐ ¿Hay campos obligatorios?
- ☐ ¿Debo controlar strings vacíos o con espacios?
- ☐ ¿Existen límites de longitud?
- ☐ ¿Hay valores mínimos o máximos?
- ☐ ¿Debo validar una conversión con TryParse?
- ☐ ¿Debo comprobar si un recurso existe?
- ☐ ¿Hay datos que no pueden repetirse?
- ☐ ¿La operación depende de otro dato o relación?
- ☐ ¿Estoy devolviendo el código HTTP correcto?
- ☐ ¿El try/catch está reservado para errores inesperados?
- ☐ ¿El mensaje de error ayuda a entender qué debe corregirse?

16. Ejercitación breve

Agregar las validaciones necesarias a los siguientes casos. No es necesario crear una base de datos; pueden utilizar listas en memoria.

Control de acceso a un evento: Registrar el ingreso de una persona. Validar id válido, persona existente, que no haya ingresado previamente y que la entrada esté habilitada.

Reserva de un simulador: Crear una reserva con fecha/hora de inicio y fin. Validar datos obligatorios, fechas coherentes, usuario existente y que el simulador no tenga otra reserva superpuesta.

Torneo de videojuegos: Inscribir un equipo. Validar nombre, cantidad de jugadores, jugadores repetidos, cupo del torneo y que el equipo no esté ya inscripto.

Préstamo de equipamiento: Registrar el préstamo de una cámara o notebook. Validar usuario, equipo existente, disponibilidad y que la fecha de devolución estimada sea posterior a la fecha de entrega.

17. Resumen

En una API bien validada: los datos incorrectos se detectan antes de ejecutar la operación, el cliente recibe un código HTTP apropiado y el mensaje explica qué debe corregirse.

- Validar primero; ejecutar después.
- Usar if para reglas conocidas y try/catch para errores inesperados.
- Comprobar formato, rango, existencia, duplicados y relaciones.
- Utilizar LINQ para consultar colecciones cuando simplifica la validación.
- Aprovechar DTOs y Data Annotations para reglas simples y repetitivas.
- Mantener los Controllers claros y, al avanzar con capas, trasladar reglas de negocio a la lógica.